CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G4
System Integration and Architecture

System Design Document (SDD)

Project Title: MedLink

Prepared By: Sumarago, Josan B.

Version: 1.0

Date: March 1, 2026

Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	[Date]	[Your Name]	Initial draft	Draft
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 10

4.0 SYSTEM ARCHITECTURE 11

5.0 API CONTRACT & COMMUNICATION 12

6.0 DATABASE DESIGN 16

7.0 UI/UX DESIGN 18

8.0 PLAN 32

EXECUTIVE SUMMARY

1.1 Project Overview

MedLink is a healthcare appointment and consultation management system that enables patients to book medical appointments, select doctors, upload medical records, and securely pay consultation fees online.

The system integrates:

- Spring Boot backend REST API
- React web application
- Android mobile application
- PostgreSQL database
- Stripe sandbox payment gateway
- Google OAuth authentication
- SMTP email notifications
- External API integration

MedLink ensures secure authentication using JWT and enforces role-based access control across all endpoints and user interfaces.

1.2 Objectives

- Develop a secure appointment booking system
- Enable patients to choose doctors and schedules
- Implement online payment confirmation for appointments
- Integrate Google OAuth login
- Implement file upload for medical documents and prescriptions
- Send automated email notifications
- Integrate a real public external API
- Provide web and Android mobile access to the same backend

1.3 Scope

Included Features:

- User Registration & Login (JWT)
- Google OAuth Login
- Role-Based Access Control
- Appointment Management (Full CRUD)
- Online Payment Integration (Stripe Sandbox)
- File Upload (Medical Documents & Prescriptions)
- External API Integration
- Email Notifications via SMTP

- Android Mobile Application (Kotlin + XML)
- PostgreSQL relational database

Excluded Features:

- Real insurance processing
- Video consultations
- SMS notifications

1.0 INTRODUCTION

1.1 Purpose

This document provides the complete architectural and technical design specification for MedLink, ensuring all system components comply with IT342 required system features and integration standards.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: MedLink
Domain: Healthcare
Primary Users: Patients, Doctors, Administrators

Problem Statement: Manual appointment scheduling leads to inefficiency, long waiting times, and lack of centralized patient record handling.
Solution: MedLink provides a centralized digital platform for appointment booking, payment confirmation, and medical document management.

2.2 Core User Journeys

Journey 1: Patient Appointment Booking

1. User registers or logs in
2. Selects doctor
3. Chooses available date and time
4. Creates appointment (PENDING_PAYMENT)
5. Completes payment via Stripe
6. Appointment becomes CONFIRMED
7. Confirmation email sent

Journey 2: Doctor Appointment Management

1. Doctor logs in
2. Views confirmed appointments
3. Conducts consultation
4. Uploads prescription file
5. Marks appointment as COMPLETED

Journey 3: Admin Management

1. Admin logs in
2. Manages doctors
3. Views users
4. Monitors payments
5. Reviews system data

2.3 Feature List (MoSCoW)

MUST HAVE

- JWT Authentication
- Role-Based Access Control
- Appointment CRUD
- Payment Integration (Stripe)
- Google OAuth
- File Upload
- Email Sending
- External API Integration
- Android App Integration

SHOULD HAVE

- Appointment filtering
- Admin analytics view

COULD HAVE

- Real-time appointment status updates (WebSocket)

WON'T HAVE

- SMS notifications
- Real insurance billing

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout, GET
- **Security:** JWT tokens, password hashing with bcrypt

Feature: Product Catalog

- **Screens:** Product Listing, Product Detail
- **Display:** Grid layout, product images, names, prices
- **Search:** By product name
- **API Endpoints:** GET /products, GET /products/{id}, GET /products/search
- **Admin Functions:** POST /products, PUT /products/{id}, DELETE /products/{id}

Feature: Role-Based Access Control

- **Screens:** Admin Dashboard, Doctor Dashboard, Patient Dashboard
- **Display:** Role-based navigation menus and restricted UI components
- **Validation:** Access restricted based on user role (PATIENT, DOCTOR, ADMIN)
- **API Endpoints:** Role-restricted endpoints (e.g., /admin/**, /doctors/**, /appointments/**)
- **Security:** API-level authorization using Spring Security roles, 403 Forbidden for unauthorized access

Feature: Appointment Management

- **Screens:** Doctor Listing, Appointment Booking Form, Appointment History, Doctor Appointment View
- **Fields:** Doctor ID, Appointment Date, Time Slot, Reason for Visit
- **Validation:** Future date only, no double booking, required fields must not be empty

- **API Endpoints:** POST /appointments, GET /appointments, GET /appointments/{id}, PUT /appointments/{id}, DELETE /appointments/{id}
- **Business Logic:** Appointment created with status PENDING_PAYMENT, updated to CONFIRMED after successful payment

Feature: Payment Integration (Stripe Sandbox)

- **Screens:** Payment Checkout Page, Payment Success Page, Payment Failure Page
- **Fields:** Appointment ID, Amount (auto-calculated from consultation fee)
- **Validation:** Payment must succeed before appointment confirmation
- **API Endpoints:** POST /payments/checkout, POST /payments/webhook
- **Security:** Stripe webhook signature verification, secure transaction ID storage, no simulated payments

Feature: Google OAuth Login

- **Screens:** Login Page (Login with Google button)
- **Fields:** Google authentication token
- **Validation:** Google token verification via backend
- **API Endpoints:** POST /auth/oauth/google
- **Security:** Backend generates its own JWT after OAuth verification

Feature: File Upload (Medical Documents & Prescriptions)

- **Screens:** Upload Medical Document, Upload Prescription
- **Fields:** File (PDF, JPG, PNG)
- **Validation:** File type restriction, file size limit, authenticated user required
- **API Endpoints:** POST /documents/upload, POST /prescriptions/upload, GET /documents/{id}
- **Security:** Files stored securely on server, file path linked to database record

Feature: External API Integration

- **Screens:** Clinic Location View / Health Information Dashboard

- **Display:** Real-time data from public API (e.g., Google Maps or public health API)
- **Validation:** API response handling and error fallback
- **API Endpoints:** GET /external/health-data
- **Security:** API keys stored securely in environment variables

Feature: Email Notifications (SMTP)

- **Screens:** Automatic (no UI interaction required)
- **Trigger:** After successful registration, after successful payment
- **Validation:** Email must be valid and registered in system
- **API Endpoints:** Triggered internally after registration/payment
- **Security:** SMTP authentication, no console-based simulation

2.5 Acceptance Criteria

AC-1: User Registration

Given I am a new user

When I enter valid email and strong password

And confirm password matches

And click "Create Account"

Then my account should be created

And I should be automatically logged in

And redirected to the homepage

AC-2: Appointment Payment

Given a patient books an appointment

When payment succeeds

Then appointment status becomes CONFIRMED

And payment record is stored

And email confirmation is sent

AC-3: Role Restriction

Given a patient

When accessing admin endpoint

Then system returns 403 Forbidden

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time must be ≤ 2 seconds for 95% of requests
- Payment processing response time must be ≤ 5 seconds (excluding external Stripe delays)
- Web page load time must be ≤ 3 seconds on broadband connection
- Android app cold start time must be ≤ 3 seconds
- System must support at least 100 concurrent users
- Database queries must complete within 500 milliseconds
- File upload processing must complete within 5 seconds for files ≤ 5MB

3.2 Security Requirements

- All communications must use HTTPS
- JWT must be required for all protected endpoints
- Passwords must be hashed using BCrypt (minimum 12 salt rounds)
- Role-based access control must be enforced at API level
- Sensitive endpoints must return 401 (Unauthorized) or 403 (Forbidden) appropriately
- Stripe webhook must validate signature before processing
- Uploaded files must be validated for type and size
- API keys (Stripe, Google OAuth, External API) must be stored in environment variables
- No plain-text passwords must be stored in the database
- DTOs must be used to prevent exposure of sensitive fields

3.3 Compatibility Requirements

Web Application

- Must support latest 2 versions of Chrome, Firefox, Edge, and Safari
- Responsive design for screen sizes:
    - Mobile (360px+)
    - Tablet (768px+)
    - Desktop (1024px+)

Mobile Application

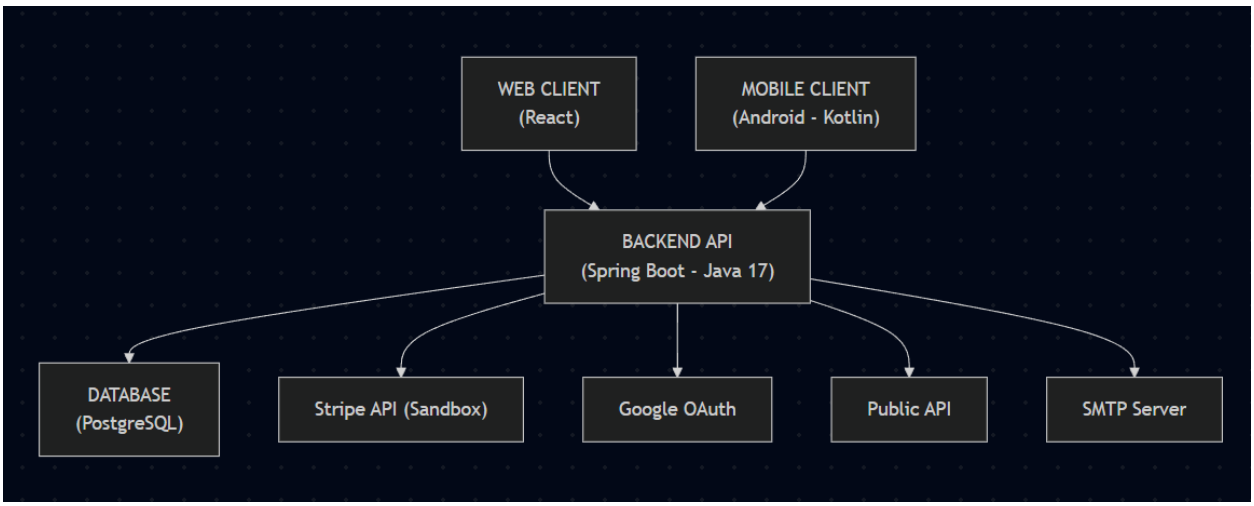
- Android API Level 34 (Android 14)
- Kotlin programming language
- XML-based UI layouts (Jetpack Compose not allowed)
- Retrofit for API communication

3.4 Usability Requirements

- A first-time user must complete appointment booking within 5 minutes
- Error messages must be clear and user-friendly
- Forms must provide validation feedback before submission
- Navigation must be consistent across pages
- Buttons must have minimum 44x44px touch size on mobile
- Role-based dashboards must clearly indicate user type

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React
- **Mobile:** Kotlin
- **Build Tools:** Maven (Backend)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** https://api.medlink.com/api/v1
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **Response Structure:**

```
json
{
  "success": boolean,
  "data": object|null,
  "error": {
    "code": string,
    "message": string,
    "details": object|null
  },
  "timestamp": string
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST /auth/register
Body: {email, password, confirmPassword, fullName}
Response: {user: {id, email, fullName, role}, token, refreshToken}

POST /auth/login
Body: {email, password}
Response: {user: {id, email, fullName, role}, token, refreshToken}

POST /auth/oauth/google
Body: {googleToken}
Response: {user: {id, email, fullName, role}, token, refreshToken}

GET /auth/me
Headers: Authorization: Bearer {token}
Response: {user: {id, email, fullName, role}}

POST /auth/logout
Headers: Authorization: Bearer {token}
Response: {message: "Logged out successfully"}

Doctor Endpoints

GET /doctors
Query: ?specialization=cardiology&page=1&limit=10
Response: {doctors: [...], pagination: {page, limit, total, pages}}

GET /doctors/{id}
Response: {doctor: {id, fullName, specialization, consultationFee, bio}}

POST /doctors (Admin only)
Body: {fullName, specialization, consultationFee, bio}
Response: {doctor: {...}}

PUT /doctors/{id} (Admin only)
Body: {specialization?, consultationFee?, bio?}
Response: {doctor: {...}}

DELETE /doctors/{id} (Admin only)
Response: {message: "Doctor removed successfully"}

Appointment Endpoints

POST /appointments
Body: {doctorId, appointmentDate, timeSlot, reason}
Response: {appointment: {id, patientId, doctorId, status, appointmentDate, timeSlot}}

GET /appointments (Authenticated)
Response: {appointments: [...]}

GET /appointments/{id}
Response: {appointment: {...}}

PUT /appointments/{id}
Body: {status}
Response: {appointment: {...}}

DELETE /appointments/{id}
Response: {message: "Appointment cancelled successfully"}

Payment Endpoints

POST /payments/checkout
Body: {appointmentId}
Response: {checkoutUrl: "https://stripe-session-url"}

POST /payments/webhook
Body: {Stripe event payload}
Response: {message: "Webhook processed"}

GET /payments/{appointmentId}
Response: {payment: {id, amount, provider, paymentStatus, transactionId}}

File Upload Endpoints

POST /documents/upload
Body: multipart/form-data (file)
Response: {document: {id, filePath, uploadedAt}}

POST /prescriptions/upload
Body: multipart/form-data (file, appointmentId)
Response: {prescription: {id, filePath}}

GET /documents/{id}
Response: File download

External API Endpoint

GET /external/health-info
Response: {data: {...}}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

```
json
{
  "success": false,
  "data": null,
  "error": {
    "code": "AUTH-001",
    "message": "Invalid credentials",
    "details": "Email or password is incorrect"
  },
  "timestamp": "2024-01-28T10:30:00Z"
}
```

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "VALID-001",
    "message": "Validation failed",
    "details": {
      "email": "Email is required",
      "password": "Must be at least 8 characters"
    }
  },
  "timestamp": "2024-01-28T10:30:00Z"
}
```

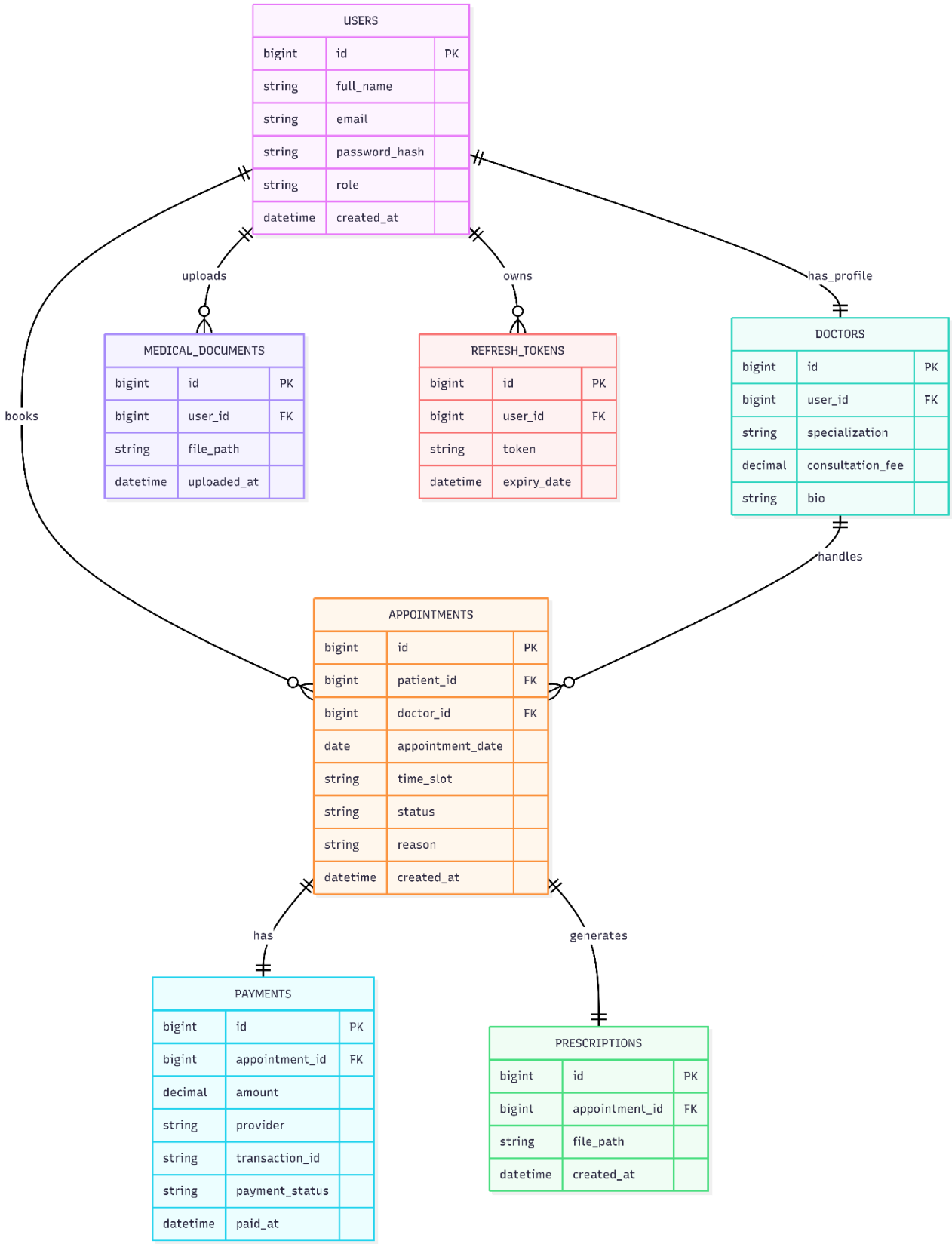

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Appointment already booked
- PAYMENT-001: Payment failed
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD



Detailed Relationships:

- **One-to-One:** User ↔ Doctor (A doctor profile belongs to exactly one user account)
- **One-to-Many:** User → Appointments (A patient can book multiple appointments)
- **One-to-Many:** Doctor → Appointments (A doctor can handle multiple appointments)
- **One-to-One:** Appointment ↔ Payment (Each appointment has one payment record)
- **One-to-One:** Appointment ↔ Prescription (Each appointment generates one prescription)
- **One-to-Many:** User → MedicalDocuments (A user can upload multiple documents)
- **One-to-Many:** User → RefreshTokens (A user can have multiple active sessions)

Key Tables:

1. **users** – User accounts and authentication
2. **doctors** – Doctor profile information
3. **appointments** – Appointment booking records
4. **payments** – Consultation payment records
5. **prescriptions** – Doctor-issued prescription files
6. **medical_documents** – Patient uploaded medical records
7. **refresh_tokens** – JWT refresh token storage

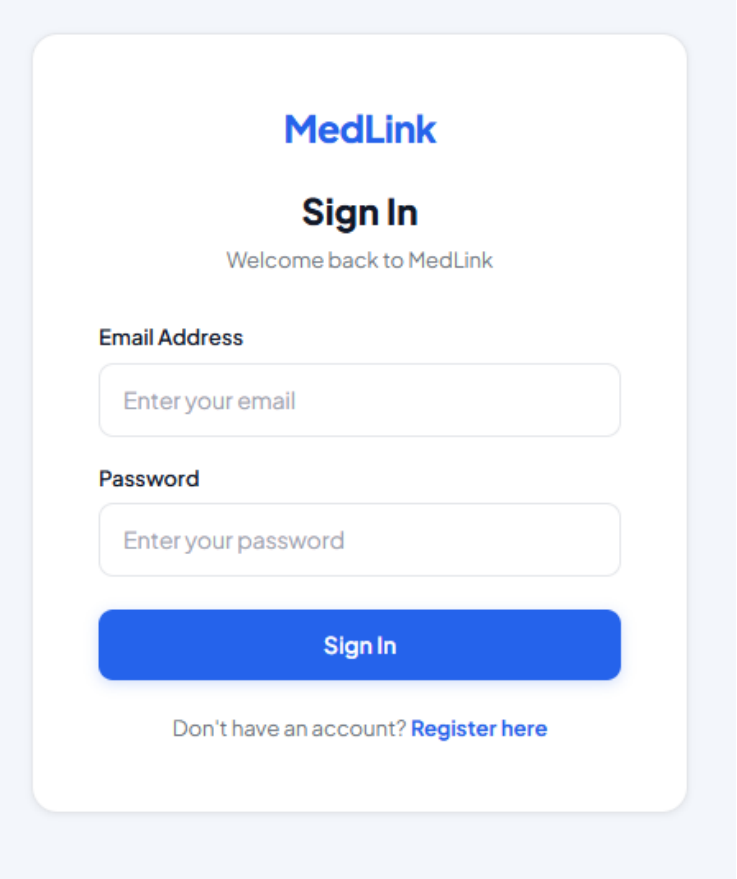
Table Structure Summary:

- **users:** id, full_name, email, password_hash, role, created_at
- **doctors:** id, user_id, specialization, consultation_fee, bio
- **appointments:** id, patient_id, doctor_id, appointment_date, time_slot, status, reason, created_at
- **payments:** id, appointment_id, amount, provider, transaction_id, payment_status, paid_at
- **prescriptions:** id, appointment_id, file_path, created_at
- **medical_documents:** id, user_id, file_path, uploaded_at
- **refresh_tokens:** id, user_id, token, expiry_date

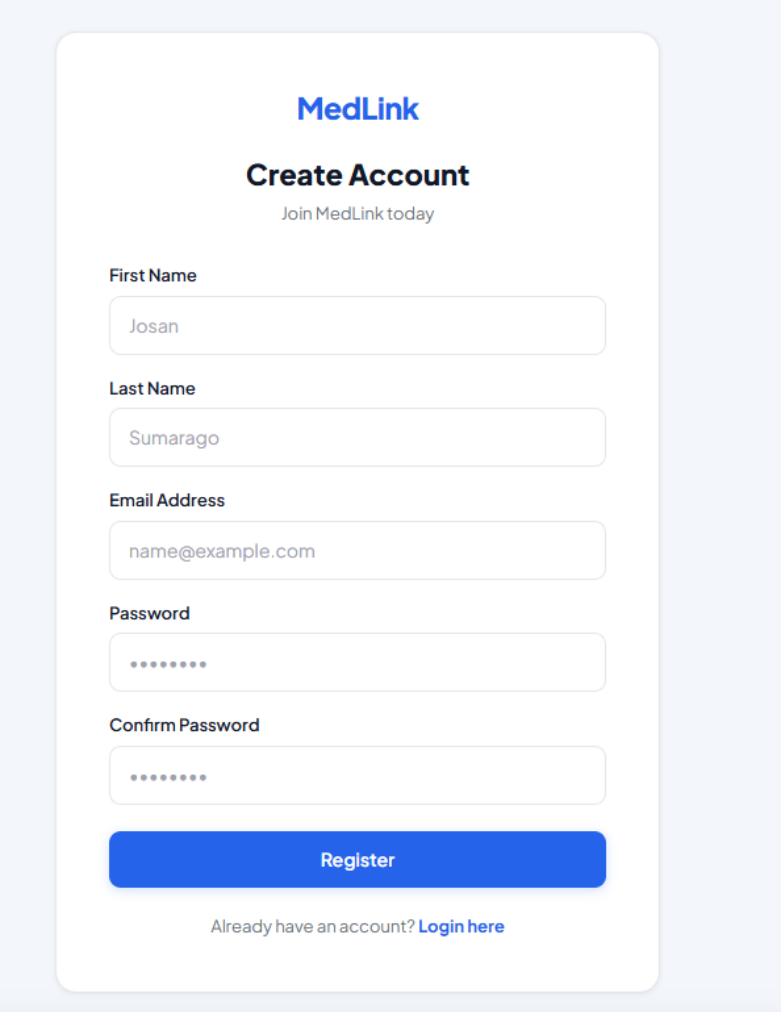
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

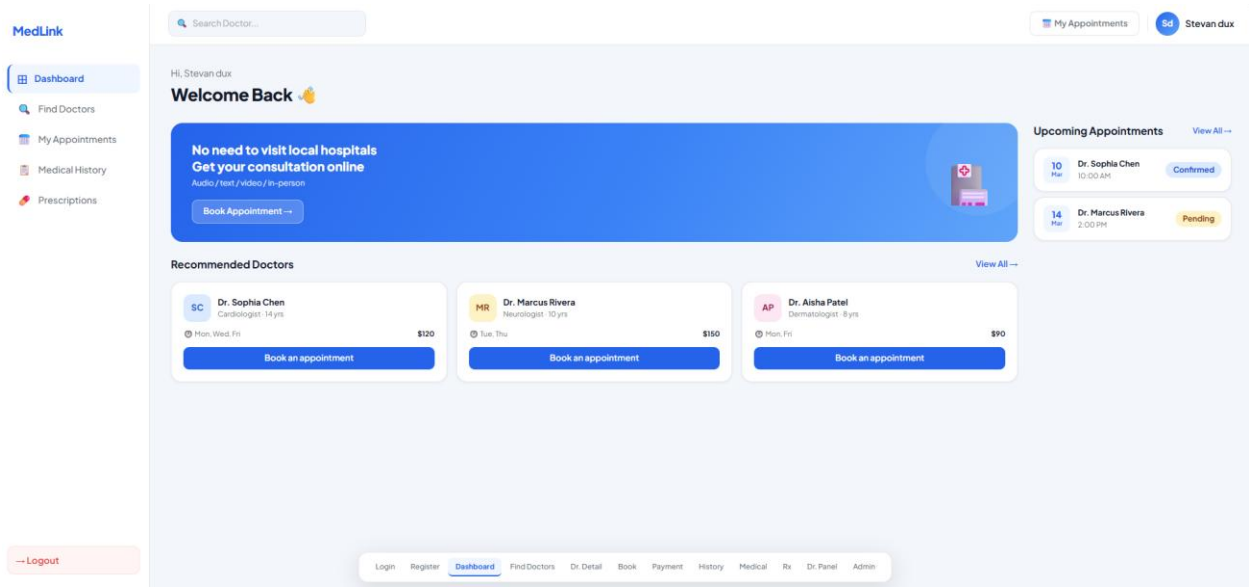
Login Page


MedLink
Sign In
Welcome back to MedLink
Email Address
Enter your email
Password
Enter your password
Sign In
Don't have an account? Register here

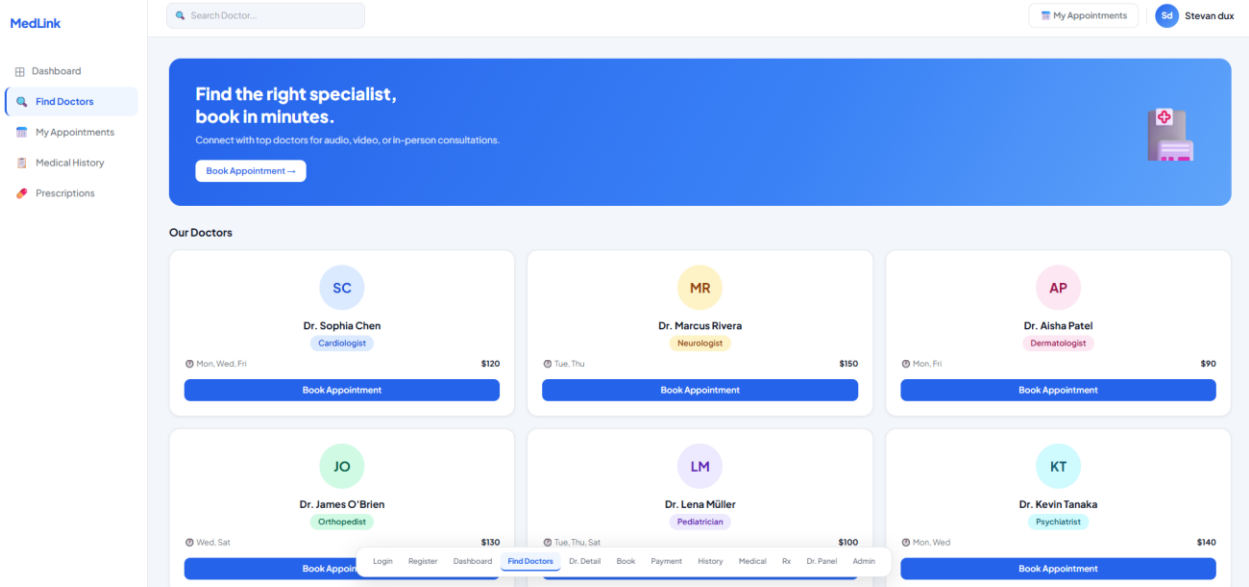
Registration Page



Dashboard



Homepage (Doctor Listing)



Doctor Detail Page

MedLink

← Back

SC

Dr. Sophia Chen

Cardiologist

Consultation Fee: $120 | Available | 14 years experience

Doctor Biography

Dr. Sophia Chen is a board-certified Cardiologist with over 14 years of clinical experience. Specializing in both preventive and interventional care, they have helped thousands of patients achieve better health outcomes. Known for a patient-first approach, evidence-based treatments, and compassionate bedside manner across multiple renowned institutions.

Available Schedule

Available on: **Mon, Wed, Fri**

9:00 AM | 10:00 AM | 11:00 AM | 2:00 PM | 3:00 PM | 4:00 PM | 5:00 PM | 6:00 PM

Book Appointment

Appointment Booking Page

MedLink

← Back

Book Appointment

SC
Dr. Sophia Chen
Cardiologist
$120 / session

Appointment Date

mm/dd/yyyy

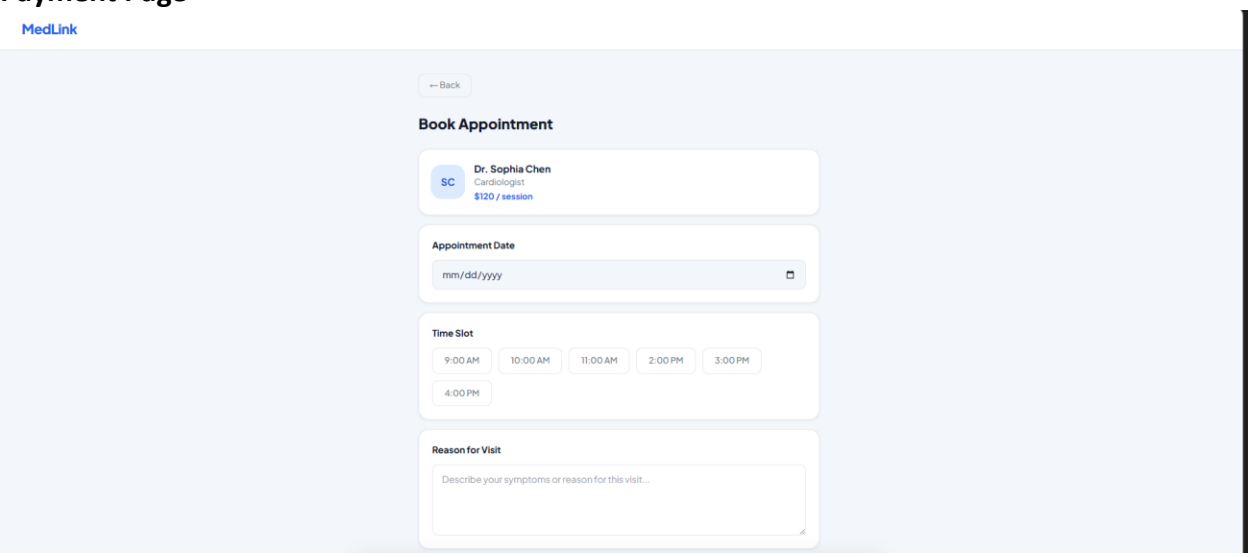
Time Slot

9:00 AM | 10:00 AM | 11:00 AM | 2:00 PM | 3:00 PM | 4:00 PM

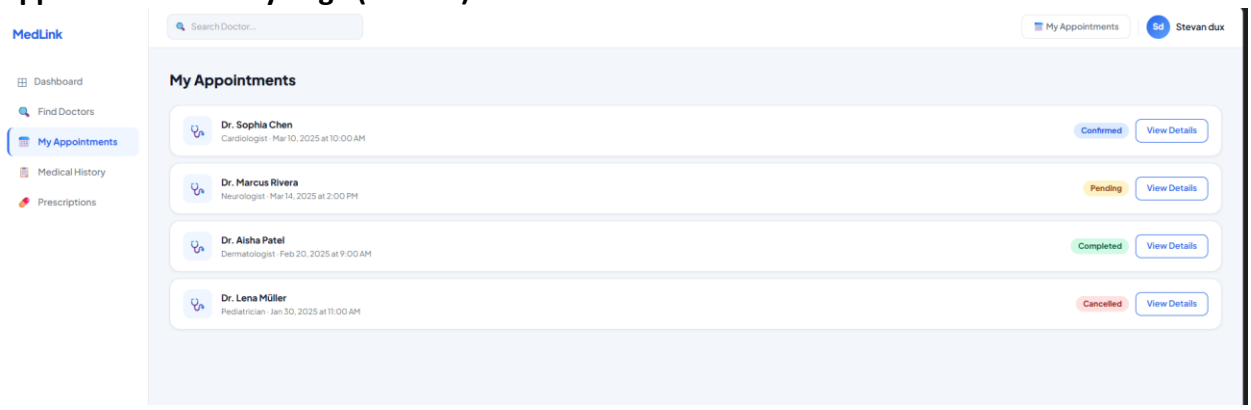
Reason for Visit

Describe your symptoms or reason for this visit...

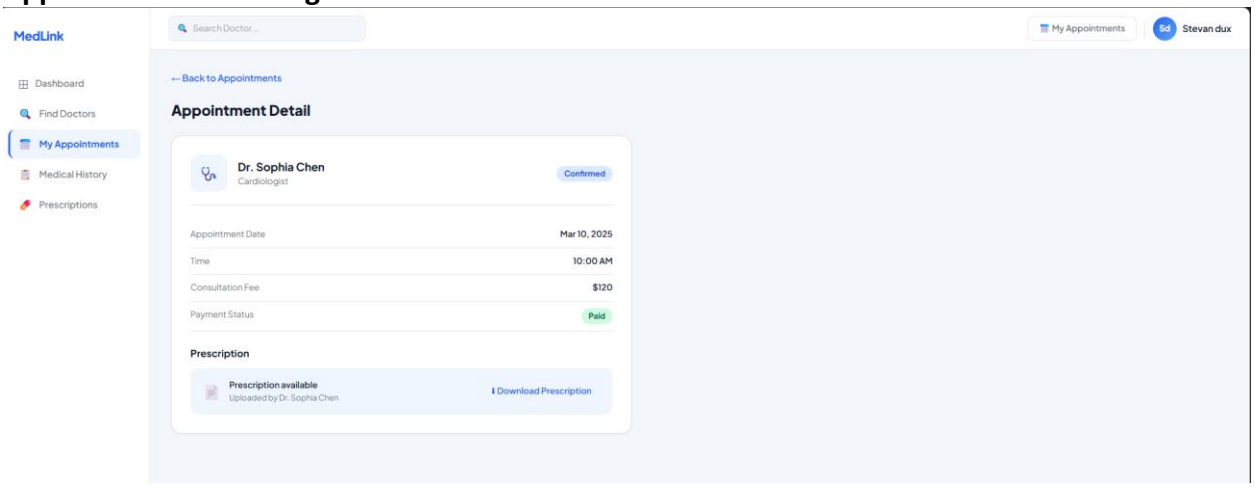
Payment Page



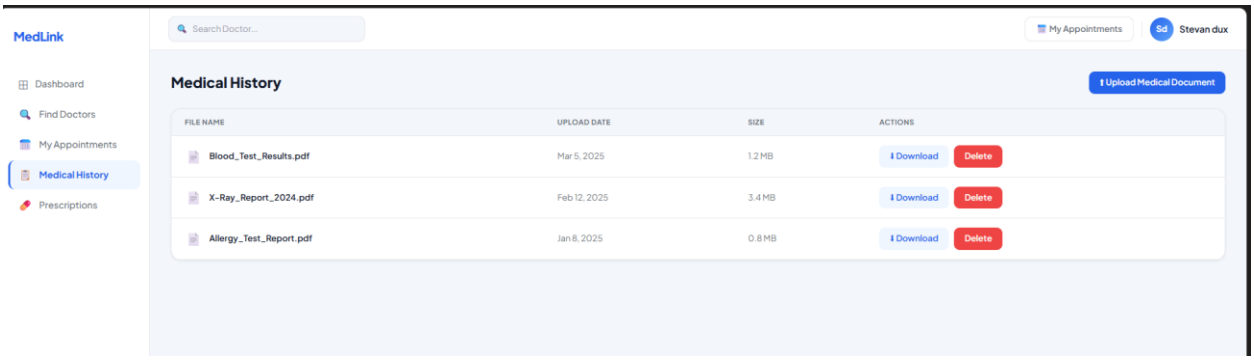
Appointment History Page (Patient)



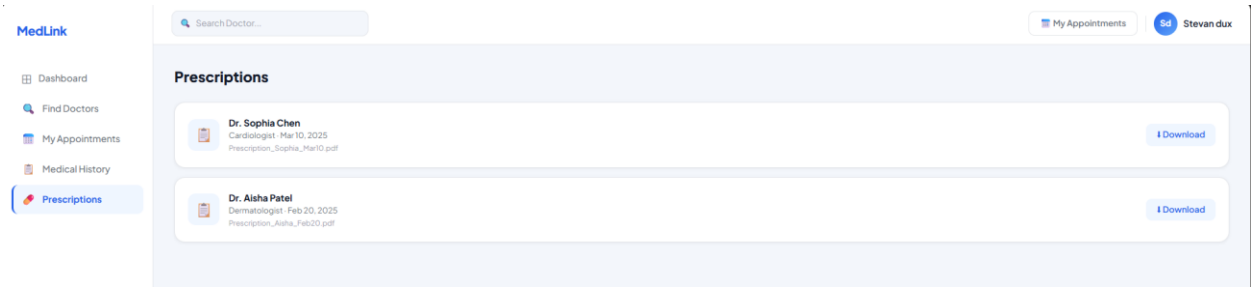
Appointment Detail Page



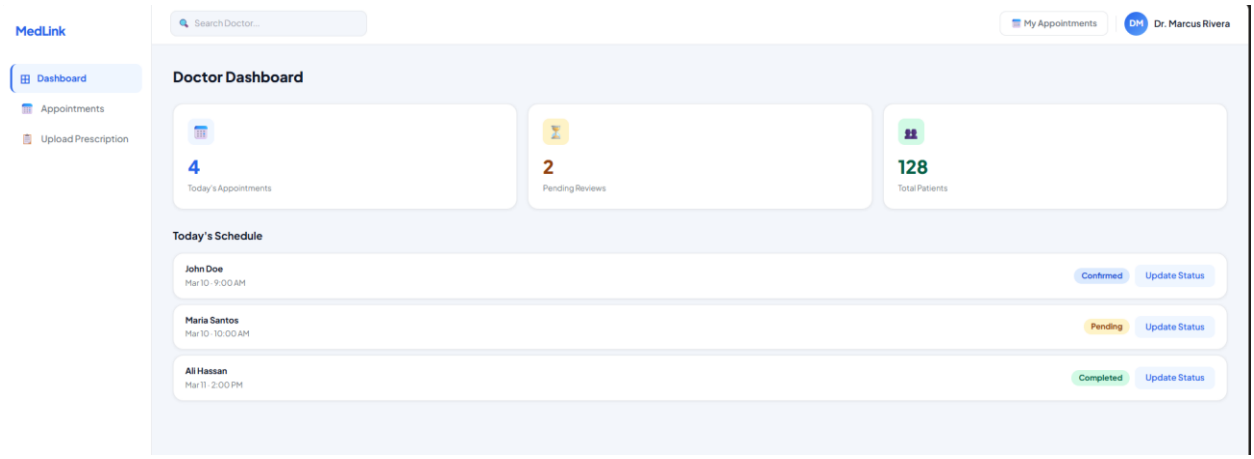
Medical History Page (Patient)



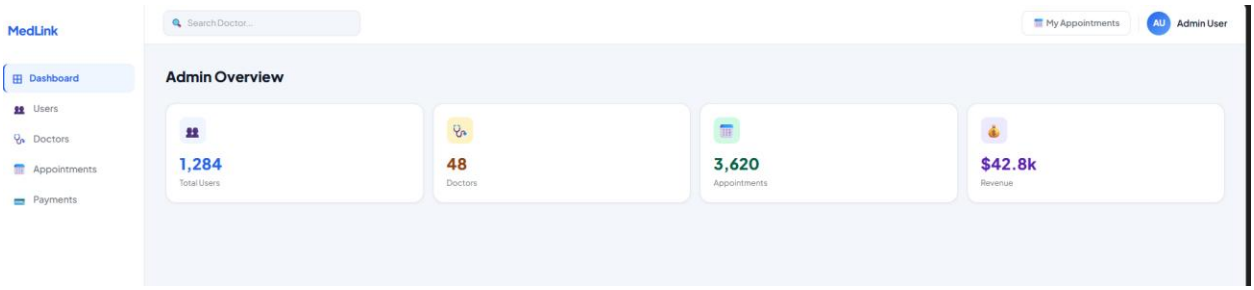
Prescriptions Page (Patient)



Doctor Dashboard

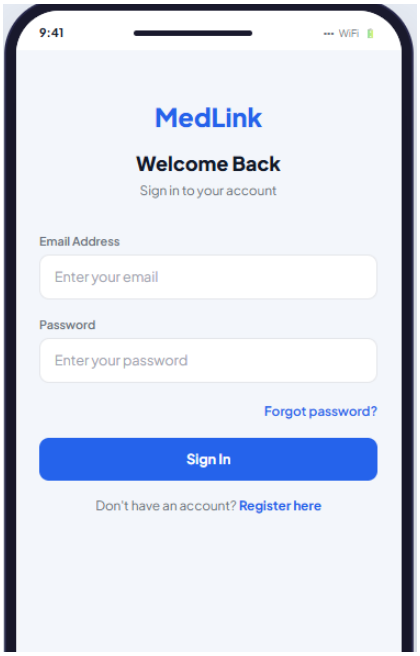


Admin Dashboard

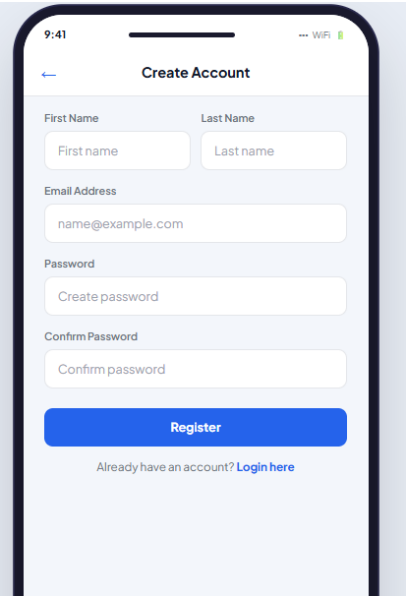


7.2 Mobile Application Wireframes

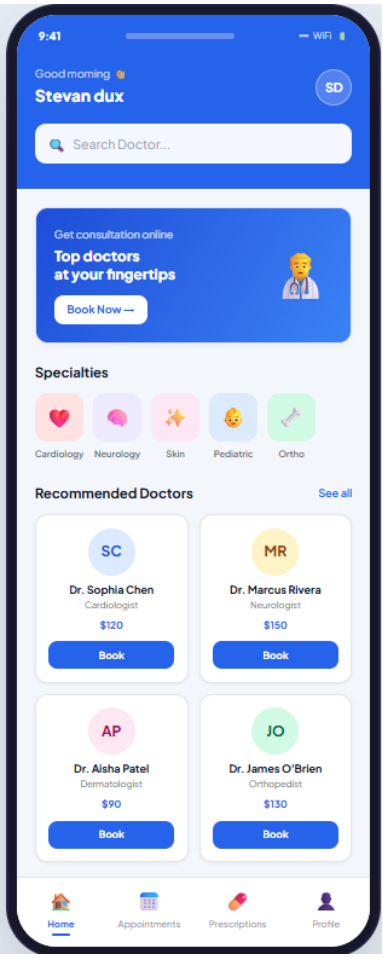
Login



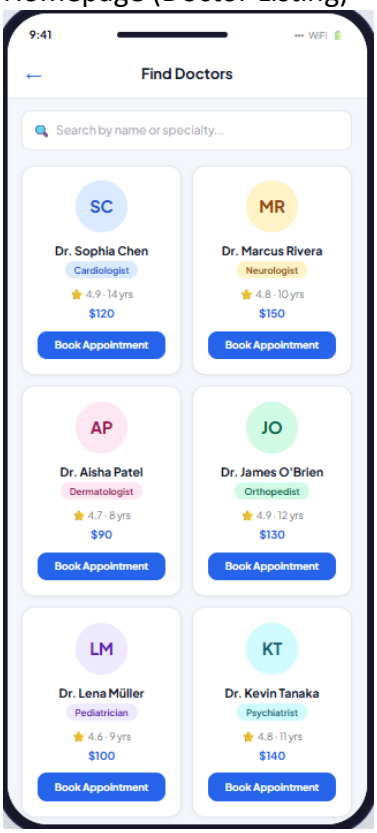
Register



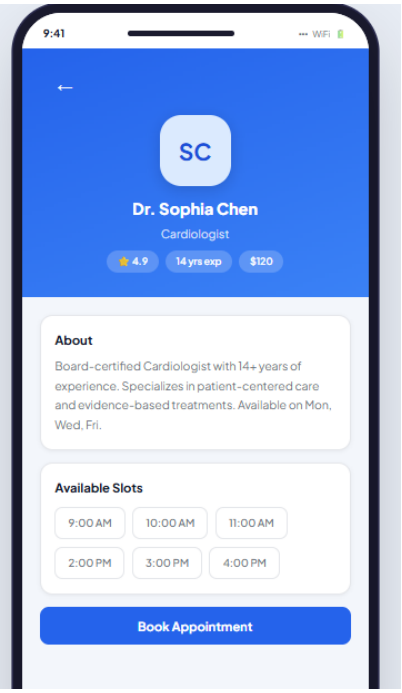
Dashboard



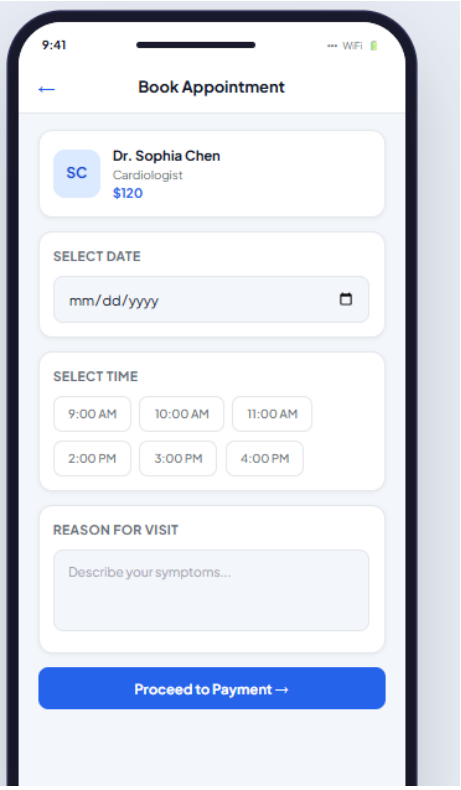
Homepage (Doctor Listing)



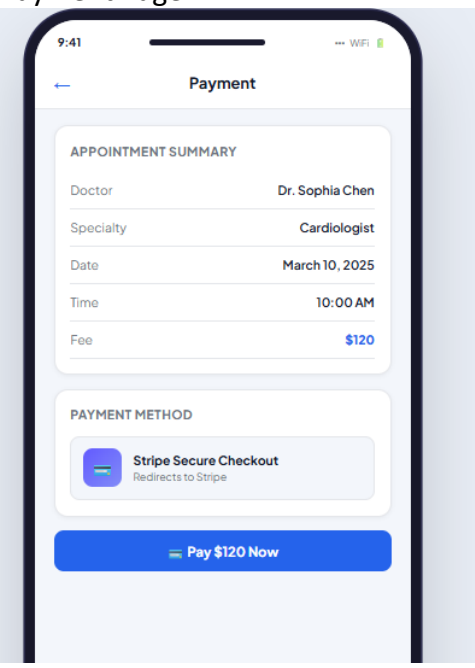
Doctor Details



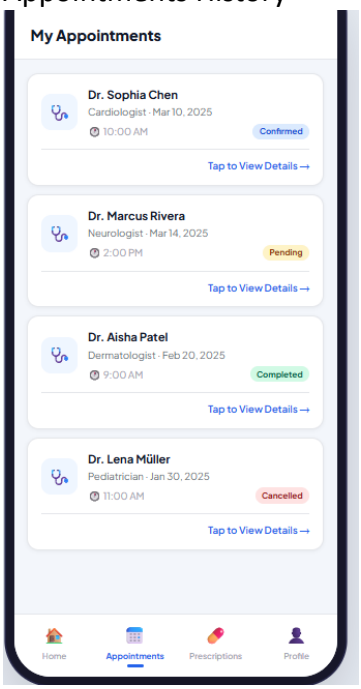
Appointment Page



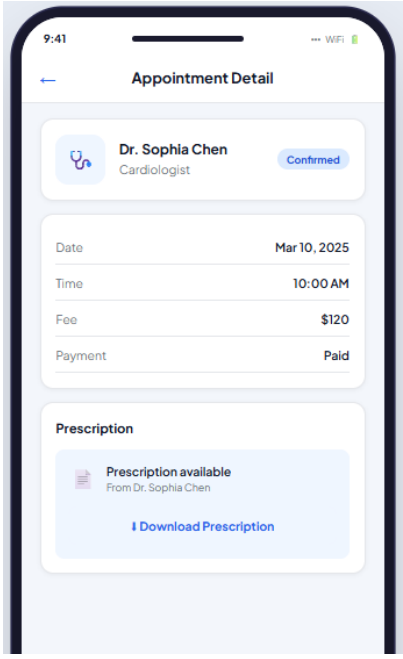
Payment Page



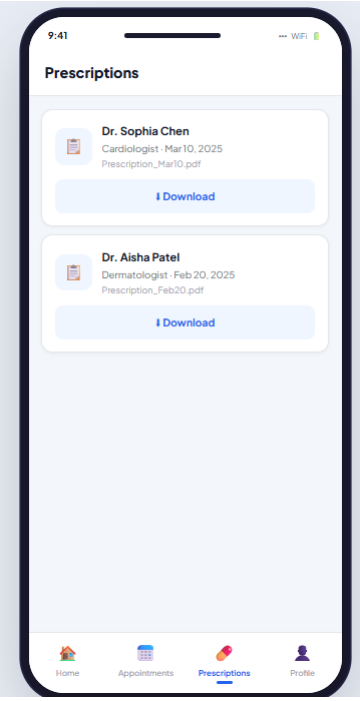
Appointments History



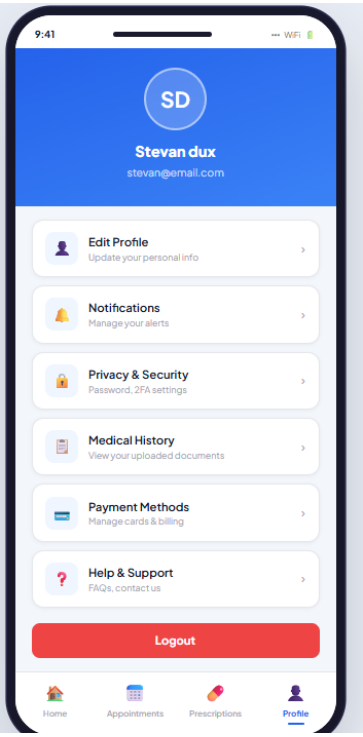
Appointment Details



Prescriptions



Profile Page



Mobile-Specific Features:

- Touch-optimized buttons (minimum 44x44px for accessibility)
- Gesture support (swipe to refresh appointments, pull-to-refresh lists)
- Offline caching for doctor profile images and basic appointment data
- Bottom navigation for main actions (Home, Appointments, Prescriptions, Profile)
- Simplified appointment booking forms optimized for mobile input
- Secure JWT token storage for authenticated sessions
- Native date picker and time picker components for better usability
- Real-time appointment status refresh after payment confirmation

Design System:

• Colors:

- Primary: #2563EB (Medical Blue)
- Secondary: #1E40AF (Dark Blue)
- Success: #10B981 (Confirmed/Completed Status)
- Warning: #F59E0B (Pending Status)
- Error: #EF4444 (Cancelled/Failed Payment)

• Typography:

- Font Family: Inter (Web), Default Android System Font (Mobile)
- Responsive font scaling for accessibility

• Spacing:

- 8px grid system for consistent layout alignment

• Components:

- Consistent button styles (Primary, Secondary, Danger)
- Standardized input fields with validation states
- Card-based layout for doctors and appointments
- Status badges with color indicators
- Modal dialogs for confirmations and alerts

• Responsive Design:

- Mobile-first approach
- Breakpoints at 640px, 768px, 1024px
- Flexible grid layout for doctor listing
- Collapsible sidebar on smaller screens

8.0 IMPLEMENTATION PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1–2)

Week 1: Requirements & Architecture

Day 1–2:
• Project setup and repository structure
• Review system requirements

Day 3–4:
• Complete Functional Requirements Specification (FRS)
• Complete Non-Functional Requirements (NFR)

Day 5–7:
• System architecture design
• Component diagram creation
• ERD diagram creation

Week 2: Detailed Design

Day 1–2:
• API contract and endpoint specification

Day 3–4:
• Database schema design
• Relationship validation

Day 5–6:
• UI/UX wireframe design (Web & Mobile)

Day 7:
• Finalize implementation plan and documentation

Phase 2: Backend Development (Week 3–4)

Week 3: Foundation

Day 1:
• Spring Boot project setup (Java 17)
• Dependency configuration

Day 2:
• PostgreSQL configuration
• Entity creation and JPA mapping

Day 3:
• JWT authentication implementation
• BCrypt password hashing

Day 4:
• Role-Based Access Control (RBAC) setup
• /auth/me endpoint implementation

Day 5:
• User and Doctor CRUD endpoints

Week 4: Core Features

Day 1:
• Appointment CRUD implementation

Day 2:
• Stripe payment integration (Sandbox)

Day 3:
• Stripe webhook handling and payment status update

Day 4:
• Google OAuth integration

Day 5:
• File upload implementation (medical documents & prescriptions)
• SMTP email integration

Phase 3: Web Application (Week 5–6)

Week 5: Frontend Foundation

Day 1:
• React project setup

Day 2:
• Authentication pages (Login, Register, OAuth login)

Day 3:
• Doctor listing page

Day 4:
• Doctor detail page

Day 5:
• Appointment booking page

Week 6: Complete Web Features

Day 1:
• Payment page integration (Stripe checkout)

Day 2:
• Appointment history page

Day 3:
• Medical history & prescription pages

Day 4:
• Admin dashboard implementation

Day 5:
• API integration testing and responsive design polish

Phase 4: Mobile Application (Week 7–8)

Week 7: Android Foundation

Day 1:
• Android Studio setup (API Level 34)
• XML layout configuration

Day 2:
• Authentication screens (Login & Register)

Day 3:
• Doctor listing screen

Day 4:
• Appointment booking screen

Day 5:
• Retrofit API service layer implementation

Week 8: Complete Mobile App

Day 1:
• Appointment history screen

Day 2:
• Prescription viewing screen

Day 3:
• Payment integration flow

Day 4:
• Error handling & UI refinement

Day 5:
• Testing on emulator and physical device
• APK generation

Phase 5: Integration & Deployment (Week 9–10)

Week 9: Integration Testing

Day 1:
• End-to-end testing (Web + Mobile + Backend)

Day 2:
• Bug fixes and validation improvements

Day 3:
• Security testing (JWT, RBAC, Webhook validation)

Day 4:
• Performance testing

Day 5:
• Final documentation updates

Week 10: Deployment

Day 1:
• Backend deployment (Railway / Render / Heroku)

Day 2:
• Web application deployment (Vercel / Netlify)

Day 3:
• Mobile APK packaging and distribution

Day 4:
• Final integration verification

Day 5:
• Final submission and defense preparation

Milestones:

• M1 (End Week 2): All documentation completed
• M2 (End Week 4): Backend fully functional with integrations
• M3 (End Week 6): Web application complete
• M4 (End Week 8): Mobile application complete
• M5 (End Week 10): Fully integrated and deployed system

Critical Path:

Authentication system implementation (Week 3)

Appointment module development (Week 4)

Payment integration (Week 4)

Web & Mobile API integration (Week 6–8)

Cross-platform testing (Week 9)

Risk Mitigation:

- Start with a minimal working version of authentication and appointment booking
- Test Stripe webhook integration early
- Validate OAuth and file upload before frontend integration
- Maintain version control backups
- Prioritize core functionality before UI enhancements